TECHNICAL REFERENCE V2.3

Shokunin
The AI Engineering Ecosystem

Multi-strategy recall, explicit session management, MCP auto-logging, and CI/CD maturity. v4.2.2 released with 99 ChromaDB entries, 25/25 healthcheck, 5 pytest tests passing.

swagger
Version 4.2.2 · 2026.05
github.com/EliasOulkadi/shokunin

Contents

Executive Summary

Shokunin is a local-first AI engineering ecosystem. It gives your agent persistent memory across sessions, 38 skills, MCP auto-logging, and a self-healing update system. Version 4.2.2 introduces session management with explicit continue, MCP server auto-logging (every tool call recorded in sessions/.jsonl), and CI/CD maturity with ruff linting and pytest integration tests.

The ecosystem has been battle-tested across 5 real days of development. 99 ChromaDB entries, 25/25 healthcheck, 5 passing pytest tests. A declarative manifest tracks 24 components with drift detection. The entire system installs with a single shell script.

Key Metrics

Metric	Value
Core scripts	17 (13 PowerShell + 4 Bash)
Python scripts	4 (chroma-helper, mcp-server, stub, read-transcript)
Managed components	24 (manifested, hash-tracked)
Skills	38 (across 8 domains)
MCP tools	8 (store, search, multi_search, consolidate, summary, list, continue, save)
LLM providers	OpenCode Go (default) + Ollama (local fallback) + NVIDIA (optional)
Memory entries	99 (growing, ~5 per session)
Healthcheck	25/25 all passing
Python tests	5 pytest (save/search, BM25 recall, date filter, session list, session continue)
CI	Ruff lint + ChromaDB + MCP + PowerShell + pytest + bash validation
Runtimes	6 (OpenCode, Claude Code, Cursor, Cline, Continue, Windsurf)
Install size	Under 200 MB (including ChromaDB + embedding model)

02 · ARCHITECTURE

Architecture

Seven layers. Each with clear boundaries. No layer depends on the layer above it. The MCP server is the single capture point for all runtimes.

Layer	Path	Responsibility
Config	`~/.config/opencode/`	Provider setup, MCP servers, agent definitions, custom commands (/save /load /status)
Instructions	`~/AGENTS.md` + `~/.claude/CLAUDE.md`	Global rules, session startup workflow, memory system instructions
Update	`~/.shokunin/`	Declarative manifest, drift detection, template-based file generation
Orchestration	`~/.shokunin/scripts/`	Session wrapping, memory persistence, healthchecks, cleanup, update engine
Memory	`~/.shokunin/memory/`	ChromaDB vector store, MCP server (auto-logs every tool call), dual-interface CLI + JSON-RPC
Skills	`~/.config/opencode/skills/` + `~/.agents/skills/`	38 SKILL.md files, automatic activation via OpenCode
Infrastructure	`~/.shokunin/backups/` + `~/.shokunin/logs/`	Automatic pre-update backups, operational logs, ChromaDB database

MCP Auto-Logging

Every call to `store_context`, `search_context`, `multi_search_context`, or `save_message` is automatically logged to `sessions/<id>.jsonl`. This is the single capture point for all runtimes. The agent does not need to explicitly save anything. The MCP server handles it transparently.

When `session continue` is called, it reads both ChromaDB entries and JSONL messages, giving the user complete context including individual conversation turns.

> The MCP server runs on demand (stdin/stdout). It never runs as a persistent daemon. Resource usage is minimal when not actively in a session.

Memory System

Multi-strategy recall with vector similarity, BM25 keyword search, temporal filtering, and reciprocal rank fusion. Auto-logged via MCP. Browseable via session list and continue.

How Recall Works

Four strategies combined when you call `recall` with a query:

1. Vector search (ChromaDB query, semantic similarity)
2. BM25 keyword search (inverted index on session markdown files, pure Python, zero dependencies)
3. Temporal filter (ISO date range, applied to both result sets)
4. Reciprocal Rank Fusion (RRF at k=60, merges and reorders combined results)

```
# Basic recall (vector + BM25 + RRF)
python ~/.shokunin/scripts/chroma-helper.py recall "Q3 budget" "shokunin" 5

# With date filter
python ~/.shokunin/scripts/chroma-helper.py recall "memory" "" 10 2026-05-14 2026-05-15
```

Memory Consolidation

The `consolidate` command groups old entries by project and extracts key terms by frequency. No LLM needed. TF-based keyword extraction.

```
python ~/.shokunin/scripts/chroma-helper.py consolidate "shokunin"
```

Dual Interface

Two interfaces share the same ChromaDB database. Both write to the same markdown fallback files. If ChromaDB is unavailable, session data persists.

Interface	Protocol	Best For
`chroma-helper.py`	CLI (stdin/stdout)	Scripting, batch operations, healthchecks, session management
`mcp-server.py`	JSON-RPC (stdin/stdout)	AI agent tool calls via MCP protocol, auto-logging

> Three-tier fallback: ChromaDB (primary), markdown files (backup), console buffer logs (emergency). If any tier fails, the next takes over silently.

Session Management

Version 4.2.2 introduces explicit session control: list recent sessions, continue any session with full context, see decisions, files, and commands parsed automatically.

Commands

```
# List recent sessions (date, project, entry count, summary)
python ~/.shokunin/scripts/chroma-helper.py session list 5

# Continue a specific session (loads all entries + decisions/files/commands)
python ~/.shokunin/scripts/chroma-helper.py session continue

# Save a message to session transcript
python ~/.shokunin/scripts/chroma-helper.py session save "message" session-xxx "user"
```

Session Continue Parsing

When loading a session via `session continue`, the system parses session_end text automatically, extracting decisions, files modified, commands executed, and checkpoints. This means even if entries were saved as a single blob, the system can still return structured data.

Field	Source	Example
`decisions_list`	Parsed from ## Decisions section	["Add BM25 search", "Fix healthcheck timing"]
`files_list`	Parsed from ## Files section	["chroma-helper.py: +recall function"]
`commands_list`	Parsed from ## Commands section	["git push to master"]
`jsonl_count`	Read from sessions/.jsonl	42 auto-logged messages

Session List Filtering

The session list automatically filters out healthcheck, test, MCP-test, and consolidated entries. Only real sessions with session_end types are shown by default. Sessions are sorted by most recent first, with session_end entries prioritized.

MCP Tools & Auto-Logging

8 MCP tools across 6 runtimes. The MCP server auto-logs every tool call to JSONL. No agent coordination needed.

MCP Tools

Tool	Description	Since	Auto-Log
`store_context`	Save text with type, tags, project, session_id	v4.0	✅
`search_context`	Vector similarity search with filters	v4.0	✅
`multi_search_context`	Vector + BM25 + temporal + RRF fusion	v4.2.1	✅
`get_session_summary`	Summarize all entries in a session	v4.0	—
`consolidate_memories`	Group old entries by project	v4.2.1	—
`list_sessions`	List recent sessions with metadata	v4.2.2	—
`continue_session`	Load full session context	v4.2.2	—
`save_message`	Record message in session transcript	v4.2.2	✅

6 Supported Runtimes

Runtime	MCP	Config
OpenCode	Native via .pack/opencode.json	Built-in + /save /load /status commands
Claude Code	Via .pack/templates/claude-code.json	Copy template
Cursor	Via .pack/templates/cursor-mcp.json	Copy to .cursor/
Cline	Via .pack/templates/cline-settings.json	Add to VS Code settings.json
Continue.dev	Via .pack/templates/continue-config.yaml	Copy to .continue/
Windsurf	Via .pack/templates/windsurf-mcp.json	Copy template

Hindsight Comparison

Hindsight (vectorize-io/hindsight) is the most accurate agent memory system on benchmarks (91.4% LongMemEval). We evaluated it after community feedback. Some ideas directly improved Shokunin. Others we chose not to adopt, for reasons grounded in our design goals.

What We Adopted

Hindsight Feature	Shokunin Implementation
Multi-strategy recall	BM25 + vector search + temporal filter + RRF fusion
Reciprocal Rank Fusion	RRF at k=60 for merging result sets
Memory consolidation	TF-based keyword extraction, no LLM

What We Skipped

Feature	Why
LLM per retain() call	Adds cost, latency, external API dependency. Shokunin stores raw text instantly without any LLM step.
PostgreSQL + pgvector	Server requirement. ChromaDB gives file-based vector storage at zero configuration.
Graph retrieval	Human-scale session memory doesn't benefit from graph traversal the way enterprise workloads do.
Cross-encoder reranking	Low priority at current scale (~100 entries). Will be added when entry count exceeds 500.
Docker / Helm / K8s	Single shell script install. Container orchestration is opposite of our design goal.

Side-by-Side

Aspect	Shokunin v4.2.2	Hindsight v0.6.2
Vector storage	ChromaDB (file-based, 30 MB)	PostgreSQL + pgvector
Keyword search	BM25 (in-memory, zero deps)	BM25 (PostgreSQL FTS)
Retrieval strategies	2 (vector + BM25) + temporal	4 (semantic, BM25, graph, temporal)
Result fusion	Reciprocal Rank Fusion	RRF + cross-encoder reranking
Entity extraction	Manual tags	LLM-based, automatic
LLM dependency	None for memory operations	Required per retain
Install complexity	1 shell script	Docker or pip + PostgreSQL
Offline capability	Full (no network needed)	Partial (needs LLM)
MCP auto-logging	Built-in (every tool call logged)	Not available
Session management	list/continue/save with text parsing	Not available
Testing	5 pytest + 25/25 healthcheck	Self-reported
Python linting	ruff (E/F/W/I checks)	ruff (via pyproject.toml)

> Hindsight is better for enterprise agents needing state-of-the-art accuracy at scale. Shokunin is better for individual developers wanting persistent AI memory without servers, API costs, or setup friction.

Commands & CI/CD Maturity

OpenCode Custom Commands

Three commands available via the TUI. They trigger the agent to perform specific memory operations.

Command	What it does
`/save`	Lists recent sessions, then saves the current session as a structured session_end (with ## Decisions, ## Files, ## Commands sections)
`/load`	Lists recent sessions with metadata, asks which to continue, loads full context with decisions/files/commands
`/status`	Runs full healthcheck (25/25) and ecosystem drift detection (24 components)

CI/CD Pipeline

Three GitHub Actions workflows run on every push to master:

Workflow	Jobs	What it validates
ci.yml	validate	SKILL.md structure, JSON config, bash scripts, CLAUDE.md template
memory-tests.yml	lint, test-python, test-scripts, test-linux, validate-config	Ruff lint, ChromaDB save/search, MCP tools, PowerShell parse, bash syntax, JSON validity, pytest integration tests, CLAUDE.md contents
release.yml	release (on tag push)	Generates GitHub Release with changelog and archive

Python Test Suite

5 pytest integration tests verify core functionality:

Test	What it verifies
`test_save_search`	Save entry to ChromaDB, then search and find it by session_id
`test_recall_bm25`	Save with unique keyword, then BM25 recall finds it via literal match
`test_recall_date_filter`	Save and verify date-filtered recall returns recent entries
`test_session_list`	Session list returns valid JSON array
`test_session_continue`	Session continue returns context with decisions field

Python Linting (ruff)

All Python code is checked with rules E (pycodestyle errors), F (pyflakes), W (warning), I (import ordering). Current status: 0 errors across all .pack/scripts/, .pack/memory/, and tests/ files.

Release Process

Create a new version with a single command:

```
git tag v4.2.3 -m "description"
git push origin v4.2.3
```

This triggers `release.yml` which generates a GitHub Release with changelog from commit messages and a tar.gz archive of skills + .pack/ + installers.

Skills Ecosystem

38 skills across 8 domains. Each is a self-contained SKILL.md file validated by CI.

Domain	Count	Skills
Infrastructure	5	docker, kubernetes, terraform, ci-cd, db-admin
Backend	4	api-forge, auth-architect, db-sculptor, error-handler
Frontend	6	component-forge, responsive-engine, motion-craft, landing-craft, ui-ux-pro-max, aesthetic-web
Mobile	2	flutter, react-native
Quality	3	test-commander, performance-profiler, code-review
Content	6	communication, content-marketing, business-proposals, seo-geo, translate-craft, documentation
Productivity	8	git-workflow, windows-powershell, runbook-gen, strategy, design, finance, legal-counsel, whendone-plus
Documents	4	kami, portfolio-auto, shokunin-update, chromadb

Skill Structure

Every skill follows the same anatomy. The YAML frontmatter contains name, description, and metadata for auto-discovery by OpenCode. The body has five mandatory sections:

1. Workflow with decision tables and step-by-step instructions
2. Error handling table (cause, fix)
3. Production checklist for pre-deployment validation
4. Anti-patterns with corrections
5. Cited sources for attribution

The Docker skill is 6,300 words covering multi-stage builds, BuildKit cache optimization, distroless bases, seccomp profiles, and CVE scanning via Trivy. The auth skill references OWASP directly. The database one has real EXPLAIN ANALYZE output. Each skill is an engineering guide for its domain, not a generic prompt.

> Skills are validated by CI on every push: YAML frontmatter, workflow, error handling, and sources must be present. The CI runs on all 38 skills.

Skill Discovery

OpenCode discovers skills from `~/.config/opencode/skills/`, `~/.agents/skills/`, and `~/.claude/skills/`. They are listed in the skill tool description and loaded on demand when the agent needs domain knowledge.

Benchmarks

Measured on 15 queries (3 iterations x 5 queries) against 120+ real ChromaDB entries using all-MiniLM-L6-v2 embeddings, in-memory BM25 (k1=1.5, b=0.75), and RRF at k=60. Both methods achieve 100% relevant-hit rate.

Metric	Vector-only	Multi-strategy
Probe hit rate	100%	40%
Any relevant hit	100%	100%
Real session hits	0%	60%
Avg latency	1,293ms	1,471ms (+13.8%)
Embedding model	all-MiniLM-L6-v2 (22M params, 384-dim)	
BM25	in-memory Python TF-IDF (k1=1.5, b=0.75)	
Fusion	RRF at k=60 (reciprocal rank fusion)	

The multi-strategy finds relevant real-world sessions that vector-only never returns. While the synthetic probe hit rate is lower, this is because BM25 correctly ranks semantically richer real entries above the probe. 100% of queries return at least one relevant result.

> Full methodology and results at github.com/EliasOulkadi/shokunin/tree/master/benchmarks. Run `python benchmarks/benchmark_memory.py` to reproduce.